

Análisis amortizado

Algoritmos y Estructura de Datos

Análisis Amortizado

Análisis de peor caso: Determinar el tiempo de ejecución en el peor de los casos de una operación de estructura de datos en función del tamaño de entrada n .

- Inserción en un conjunto implementado sobre arreglo dinámico tiene costo $O(n)$ cuando se expande el arreglo. Pero es pesimista si consideramos que hasta llenar el arreglo el costo es $O(1)$.

Análisis amortizado: Determinar el tiempo de ejecución en el peor de los casos de una secuencia de m operaciones de estructura de datos.

- ¿Cuánto cuesta la inserción teniendo en cuenta que es cara solo después de varias inserciones baratas? ¿ $O(m)$?
- De `java.util. ArrayList<E>`:

Each `ArrayList` instance has a *capacity*. The capacity is the size of the array used to store the elements in the list. It is always at least as large as the list size. As elements are added to an `ArrayList`, its capacity grows automatically. The details of the growth policy are not specified beyond the fact that adding an element has constant amortized time cost.

Aplicaciones de análisis amortizado

- Splay trees.
- Dynamic tables (ej: array dinámicos, tablas de hash)
- Fibonacci heaps.
- Garbage collection.
- Move-to-front list updating.
- ...

Multi pop stack

Operaciones de soporte en un conjunto de elementos:

- PUSH(S, x): añade el elemento x a la pila S.
- POP(S): elimina y devuelve el elemento añadido más recientemente.
- MULTI-POP(S, k): elimina los elementos k añadidos más recientemente.

Teorema: A partir de una pila vacía, cualquier secuencia entremezclada de n operaciones PUSH, POP y MULTI-POP cuesta $O(n^2)$.

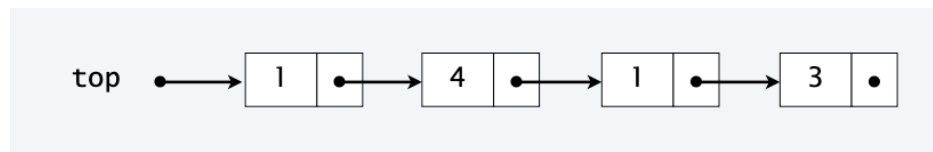
Prueba:

Usando una lista enlazada:

POP y PUSH tardan $O(1)$ tiempo cada uno.

MULTI-POP tarda $O(n)$ tiempo.

Límite superior muy pesimista



Multi pop stack

Operaciones de soporte en un conjunto de elementos:

- PUSH(S, x): añade el elemento x a la pila S.
- POP(S): elimina y devuelve el elemento añadido más recientemente.
- MULTI-POP(S, k): elimina los elementos k añadidos más recientemente.

Teorema: A partir de una pila vacía, cualquier secuencia entremezclada de n operaciones PUSH, POP y MULTI-POP cuesta $O(n)$.

Prueba:

- Un elemento se extrae como máximo una vez por cada vez que se inserta.
- Hay $\leq n$ operaciones PUSH.
- Por lo tanto, hay $\leq n$ operaciones POP (incluidos los realizados dentro de MULTI-POP).

Conclusión: el costo amortizado por operación es $O(1)$



Método del Banquero

- Asignar a cada operación un costo amortizado $\hat{c}$
 - Puede ser distinta que el costo peor caso c
- Probar que para cualquier secuencia de longitud m vale:

$$\sum_{i=0}^m \hat{c}_i \geq \sum_{i=0}^m c_i$$

Alternativamente, que siempre tenemos crédito positivo:

$$\sum_{i=0}^m \hat{c}_i - \sum_{i=0}^m c_i \geq 0$$

- Calcular una cota $T(m)$ para el costo amortizado de una secuencia de longitud m . Buscamos peor caso, es decir la secuencia más cara.
- Calcular el costo por operación como $T(m)/m$

Pila multipop: método del banquero

Créditos: 1 crédito paga por un PUSH o un POP.

Invariante: Cada elemento de la pila tiene 1 crédito.

Contabilidad.

- PUSH(S, x): carga 2 créditos.
 - 1 crédito para pagar por el push de x ahora
 - 1 crédito para pagar para pagar por el futuro pop más adelante
- POP(S): carga 0 créditos.
- MULTIPOP(S, k): carga 0 créditos.

Teorema:. A partir de una pila vacía, cualquier secuencia entremezclada de n operaciones PUSH, POP y MULTI-POP tarda $O(n)$ tiempo.

Prueba:

- El número de créditos en la estructura de datos ≥ 0 .
- Costo real total de n operaciones $\leq$ suma de los costos amortizados $\leq 2n$.

Método Potencial

- Asignar a **estado** un crédito o valor potencial: $\phi(D)$
- Podemos pensar el costo amortizado de una operación como:

$$\hat{c}_i = c_i + \phi(D_i) - \phi(D_{i-1})$$

- Para una secuencia de longitud n tenemos:

$$\sum_{i=0}^n \hat{c}_i = \sum_{i=0}^n c_i + \phi(D_i) - \phi(D_{i-1}) = \left(\sum_{i=0}^n c_i \right) + \phi(D_n) - \phi(D_0)$$

- Debemos probar que $\phi(D_n) - \phi(D_0)$ es mayor que cero para cualquier secuencia.
- Calcular una cota $T(m)$ para el costo amortizado de una secuencia de longitud n (peor caso)
- Calcular el costo por operación como $T(n)/n$

Pila Multipop: método potencial

Función de potencial: Sea $\Phi(D)$ = número de elementos actualmente en la pila.

$$\Phi(D_0) = 0.$$

$$\Phi(D_i) \geq 0 \text{ por cada } D_i.$$

Teorema: A partir de una pila vacía, cualquier secuencia entremezclada de n operaciones PUSH, POP y MULTI-POP tarda $O(n)$.

Prueba: [Caso 1: push]

- Supongamos que la i -ésima operación es un PUSH.
- El costo real $c_i = 1$.
- El costo amortizado $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 + 1 = 2$.

Pila multipop: método potencial

Teorema: A partir de una pila vacía, cualquier secuencia entremezclada de n operaciones PUSH, POP y MULTI-POP tarda $O(n)$.

[Caso 2: pop]

- Supongamos que la i -ésima operación es un POP.
- El costo real $c_i = 1$.
- El costo amortizado $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) = 1 - 1 = 0$.

[Caso 3: multi-pop]

- Supongamos que la i -ésima operación es un MULTI-POP de k objetos.
- El costo real $c_i = k$.
- El costo amortizado $\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}) =$ si $k \geq \Phi(D_{i-1}) : k - k = 0$.
si $k < \Phi(D_{i-1}) : \Phi(D_{i-1}) - \Phi(D_{i-1}) = 0$

Pila multipop: método potencial

2 para push; 0 para pop y multi-pop

Teorema: A partir de una pila vacía, cualquier secuencia entremezclada de n operaciones PUSH, POP y MULTI-POP tarda (n) .

Prueba: Igual que la anterior (los costos son los mismos)

Tabla dinámica

Almacenar elementos en una tabla (por ejemplo, para una tabla hash, un montón binario).

Dos operaciones: INSERTAR y ELIMINAR.

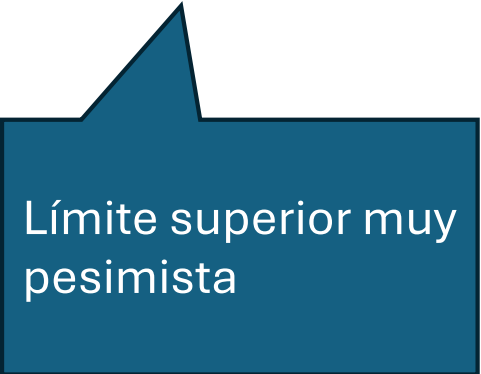
Demasiados elementos insertados $\Rightarrow$ expandir tabla.

Demasiados elementos eliminados $\Rightarrow$ contraer tabla.

Requisito: si la tabla contiene m elementos, entonces espacio = $\Theta(m)$.

Teorema. A partir de una tabla dinámica vacía, cualquier secuencia entremezclada de n operaciones INSERT y DELETE tarda $O(n^2)$ tiempo.

Prueba. Cada INSERT o DELETE tarda $O(n)$ tiempo. ■



Límite superior muy pesimista

Tabla dinámica: solo insertar

- Al insertar en una tabla vacía, asigne una tabla de capacidad 1.
- Al insertar en una tabla completa, asigne una nueva tabla del **doble** de la capacidad y copie todos los elementos.
- Inserte el elemento en la tabla.

insert	old capacity	new capacity	insert cost	copy cost
1	0	1	1	–
2	1	2	1	1
3	2	4	1	2
4	4	4	1	–
5	4	8	1	4
6	8	8	1	–
7	8	8	1	–
8	8	8	1	–
9	8	16	1	8
⋮	⋮	⋮	⋮	⋮

Tabla dinámica: solo inserción (método agregado)

Teorema. [a través del método agregado] A partir de una tabla dinámica vacía, cualquier secuencia de n operaciones INSERT tarda $O(n)$ tiempo.

Prueba: Sea c_i el costo de la i -ésima inserción.

$c_i = \begin{cases} i & \text{si } i-1 \text{ es potencia de } 2; \\ 1 & \text{sino es} \end{cases}$

A partir de una tabla vacía, el costo de una secuencia de n operaciones INSERT es:

$$\sum_{i=1}^n c_i \leq n + \sum_{j=0}^{\lfloor \lg n \rfloor} 2^j < n + 2n = 3n$$

Tabla dinámica: solo inserción (método potencial)

Teorema. A partir de una tabla dinámica vacía, cualquier secuencia de n operaciones INSERT tarda $O(n)$ tiempo.

Prueba : Sea $\Phi(D_i) = 2 \text{ tamaño}(D_i) - \text{capacidad}(D_i)$.

- $\Phi(D_i) = 0$.
- $\Phi(D_i) \geq 0$ por cada D_i .

inmediatamente después de la duplicación
capacidad $(D_i) = 2 \text{ tamaño } (D_i)$



Tamaño = 6, capacidad = 8, $\Phi = 4$

Costo amortizado $\hat{c} = c + (\Phi(D') - \Phi(D))$

El costo de la operación mas el “cambio de potencial”, (la diferencia entre el estado nuevo y el Viejo)

Tabla dinámica: solo inserción (método potencial)

Teorema: A partir de una tabla dinámica vacía, cualquier secuencia de n operaciones INSERT tarda $O(n)$ tiempo.

Sea $\Phi(D_i) = 2 \text{ tamaño}(D_i) - \text{capacidad}(D_i)$.

Caso 0. [primera inserción]

- Costo real $c_1 = 1$.
- $\Phi(D_1) - \Phi(D_0) = (2 \text{ tamaño}(D_1) - \text{capacidad}(D_1)) - (2 \text{ tamaño}(D_0) - \text{capacidad}(D_0))$
 $= 1$.
- Costo amortizado $\hat{c}_1 = c_1 + (\Phi(D_1) - \Phi(D_0))$
 $= 1 + 1 = 2$.

Tabla dinámica: solo inserción (método potencial)

Teorema: A partir de una tabla dinámica vacía, cualquier secuencia de n operaciones INSERT tarda $O(n)$ tiempo.

Sea $\Phi(D_i) = 2 \text{ tamaño}(D_i) - \text{capacidad}(D_i)$.

Caso 1. [sin expansión de la tabla]

- $\text{capacidad}(D_i) = \text{capacidad}(D_{i-1})$.

- Costo real $c_i = 1$.
- $\Phi(D_i) - \Phi(D_{i-1}) = (2 \text{ tamaño}(D_i) - \text{capacidad}(D_i)) - (2 \text{ tamaño}(D_{i-1}) - \text{capacidad}(D_{i-1}))$
 $= 2$.
- Costo amortizado $\hat{c}_i = c_i + (\Phi(D_i) - \Phi(D_{i-1}))$
 $= 1 + 2$
 $= 3$.

Tabla dinámica: solo inserción (método potencial)

Teorema: [método potencial] A partir de una tabla dinámica vacía, cualquier secuencia de n operaciones INSERT tarda $O(n)$ tiempo.

Caso 2. [expansión de la tabla]

- $\text{capacidad}(D_i) = 2 \text{ capacidad}(D_{i-1})$.

- Costo real $c_i = 1 + \text{capacidad}(D_{i-1})$.
- $\Phi(D_i) - \Phi(D_{i-1}) = (2 \text{ tamaño}(D_i) - \text{capacidad}(D_i)) - (2 \text{ tamaño}(D_{i-1}) - \text{capacidad}(D_{i-1}))$
 $= 2 - \text{capacidad}(D_i) + \text{capacidad}(D_{i-1})$
 $= 2 - \text{capacidad}(D_{i-1})$.
- Costo amortizado $\hat{c}_i = c_i + (\Phi(D_i) - \Phi(D_{i-1}))$
 $= 1 + \text{capacidad}(D_{i-1}) + (2 - \text{capacidad}(D_{i-1}))$
 $= 3$.

Tabla dinámica: solo inserción (método potencial)

Teorema: A partir de una tabla dinámica vacía, cualquier secuencia de n operaciones INSERT tarda $O(n)$ tiempo.

[poniendo todo junto]

- Costo amortizado por operación $\hat{c}_i \leq 3$.
- Costo real total de n operaciones $\leq$ suma del costo amortizado $\leq 3n$. ■